

Guía de Entrevista iOS — Swift y SwiftUI

Nicolas Soto · Mis-estudios · actualizada al 6 de octubre de 2026

Todo lo que vimos desde la Lección 1 hasta el Reto 19, en formato de entrevista.

Cómo estudiar con esta guía

1. **Tapa la respuesta con una hoja.** Lee la pregunta y **respóndela en voz alta**, como si tuvieras al entrevistador enfrente.
2. Destapa y compara. Si te faltó algo, marca la pregunta con una **X** en el margen.
3. Al día siguiente repite **solo las marcadas**. Cuando una pregunta tenga tres ✓ seguidas, ya la dominas.
4. En las partes "**Encuentra el bug**" y "**Escribe de memoria**", usa lápiz y papel. Las soluciones están al final (Parte 7).

Regla de oro de entrevista: si no sabes algo, di lo que **sí** sabes y razona en voz alta. Al entrevistador le importa cómo piensas, no solo que aciertes.

Índice

- Parte 1 — Fundamentos de Swift (Lecciones 1 a 8)
 - Parte 2 — Funciones a fondo: etiquetas, `_` y `return`
 - Parte 3 — Algoritmos estilo LeetCode (Lección 9 y retos)
 - Parte 4 — SwiftUI (Lección 10)
 - Parte 5 — Encuentra el bug
 - Parte 6 — Escribe de memoria
 - Parte 7 — Soluciones
 - Parte 8 — Lo que viene (todavía no lo vemos)
-

Parte 1 — Fundamentos de Swift

1.1 Variables y tipos

P1. ¿Cuál es la diferencia entre `let` y `var`?

`let` crea una **constante**: una vez que le das un valor, no puede cambiar. `var` crea una **variable**: su valor puede cambiar. En Swift se usa `let` por defecto, y `var` solo cuando el valor de verdad va a cambiar. Así el código es más seguro y el compilador te avisa si cambias algo por error.

```
let nombre = "Nicolas"    // nunca cambia
var edad = 25
edad = 26                 // ✅ permitido
nombre = "Luis"           // ❌ error: cannot assign to value: 'nombre' is a 'let'
                           constant
```

P2. ¿Cuáles son los tipos básicos de Swift?

`String` (texto, con comillas), `Int` (número entero), `Double` (número con decimales), `Bool` (solo `true` o `false`) y `Character` (una sola letra).

P3. ¿Qué es la inferencia de tipos?

Swift **adivina el tipo** por el valor que le das, así que no siempre hay que escribirlo. `var x = 5` → Swift sabe que es `Int`. `var texto = ""` → sabe que es `String` (vacío). Si quieres ser explícito: `var x: Int = 5`. En una lista vacía **hay que** escribir el tipo, porque no hay valor del que adivinar: `var lista: [Int] = []`.

P4. ¿Un `Bool` puede valer 10?

No. Un `Bool` solo puede ser `true` o `false`. Nada más.

P5. ¿Qué es la interpolación de strings?

Meter el valor de una variable dentro de un texto con `\( )`.

```
print("Tengo \(edad) años")    // Tengo 26 años
```

P6. ¿Por qué no se puede sumar un `Int` con un `Double` directamente?

Swift es un lenguaje de **tipado fuerte**: no convierte tipos solo. Hay que convertir a mano: `Double(miEntero) + 3.5`.

1.2 Condicionales y operadores

P7. ¿Cuál es la diferencia entre `=` y `==`?

`=` **guarda** (asigna) un valor en una caja. `==` **pregunta** si dos cosas son iguales y devuelve un `Bool`. Dentro de un `if` siempre va `==`, porque un `if` pregunta.

```
hoy = 5           // guarda 5
if hoy == 5 { }   // pregunta: ¿hoy vale 5?
```

P8. ¿Cuáles son los operadores de comparación y los lógicos?

Comparación: `==` igual, `!=` distinto, `>` mayor, `<` menor, `>=` mayor o igual, `<=` menor o igual.
Lógicos: `&&` = **y** (las dos deben cumplirse), `||` = **o** (basta una), `!` = **no** (invierte).

P9. ¿Qué diferencia hay entre `if / else if / else` y dos `if` sueltos?

`if ... else if ... else` es **una sola decisión** con varios caminos: entra solo en el primero que se cumpla. Dos `if` sueltos, uno debajo del otro, son **dos preguntas distintas**: se revisan las dos.

P10. ¿Todo `if` necesita un `else`?

No. El `else` es la excepción, no la regla. Y solo puede ir **pegado al `}` de un `if`**, nunca a un `for` ni a un `return`.

1.3 Loops y rangos

P11. ¿Cuál es la diferencia entre `1...10` y `1..<10`?

`1...10` es un **rango cerrado**: incluye el 10. `1..<10` es un **rango abierto**: llega hasta el 9. Para recorrer las posiciones de un array se usa `0..lista.count`, porque las posiciones empiezan en 0.

P12. ¿Qué pasa si escribes `2...1`?

La app **se cae** (crash): *Fatal error: Range requires lowerBound <= upperBound*. El inicio de un rango no puede ser mayor que el final.

P13. ¿Cuál es la diferencia entre `for` y `while`?

`for` se usa cuando **sabes cuántas vueltas** vas a dar (recorrer una lista, un rango). `while` se usa cuando **no sabes cuántas**: repite **mientras** una condición sea verdadera.

P14. ¿Qué es un loop infinito y cómo se evita?

Un `while` cuya condición nunca se vuelve falsa, así que nunca termina. Se evita cambiando **dentro del loop** algo que acerque la condición a falso (por ejemplo `n = n / 10`).

P15. ¿Qué hace el operador `%`?

Da el **resto** de una división. `7 % 2 = 1`. `numero % 2 == 0` → el número es par. `numero % 10` → da el último dígito.

P16. ¿Cuánto da `123 / 10` en Swift? ¿Por qué?

Da `12`, no `12.3`. Los dos son `Int`, así que es **división entera**: corta los decimales. Con `Double` (`123.0 / 10.0`) sí da `12.3`.

P17. ¿La variable del `for` necesita `var` o `let`?

No. El `for` la crea solo (`for i in 1...5`) y solo existe dentro de las llaves del loop.

1.4 Arrays y diccionarios

P18. ¿Cuál es la diferencia entre un Array y un Diccionario?

Array: lista **ordenada**. Se accede por **posición** (índice, empieza en 0): `notas[0]` . **Diccionario:** pares **clave: valor**. Se accede por **nombre**: `contacto["nombre"]` .

P19. ¿Qué devuelve un diccionario cuando le pides una clave? ¿Por qué?

Devuelve un **Optional**, porque la clave podría no existir. Por eso se usa `?? "sin valor"` o `if let` .

P20. ¿Qué pasa si pides `lista[10]` y la lista tiene 5 elementos?

Crash: *Index out of range*. Swift no te deja leer fuera de la lista.

P21. ¿Qué hacen `.count` , `.append()` , `.isEmpty` y `Array(repeating:count:)` ?

- `.count` → cuántos elementos tiene
- `.append(x)` → agrega `x` al final
- `.isEmpty` → `true` si está vacía
- `Array(repeating: false, count: 66)` → crea una lista de 66 cajitas, todas con `false` (la caja de huevos 🥚)

1.5 Optionals

P22. ¿Qué es un Optional?

Una caja que **puede tener un valor o estar vacía** (`nil`). Se declara con `?` después del tipo: `var apellido: String? = nil` . Swift te **obliga** a revisar si está vacía antes de usarla. Así evita crashes.

P23. ¿Qué es `nil` ?

La ausencia de valor ("la caja está vacía"). Solo un Optional puede ser `nil` .

P24. ¿Cuáles son las formas de abrir un Optional? ¿Cuál es peligrosa?

1. `if let` (seguro): entra solo si hay valor.
2. `??` (seguro): da un valor por defecto si está vacío.
3. `!` (peligroso): fuerza a abrirla. Si está vacía, **crash**. Se evita.

```
if let a = apellido { print(a) } else { print("Sin apellido") }
print(apellido ?? "Sin apellido")
print(apellido!) // 🌟 crash si es nil
```

P25. ¿Qué significa `if let datos = ..., let guardados = ...`?

Dos `if let` en uno: entra **solo si los dos** tienen valor. Si cualquiera es `nil`, se salta el bloque.

1.6 Structs y classes

P26. ¿Cuál es la diferencia entre `struct` y `class`?

- `struct` = tipo por **VALOR**. Al copiarlo se crea una **copia independiente**. Es como una **fotocopia**: si rayas la tuya, la mía no cambia.
- `class` = tipo por **REFERENCIA**. Al copiarla, las dos variables apuntan **al mismo objeto**. Es como un **Google Doc compartido**: si tú escribes, yo lo veo.
- El `struct` crea el `init` solo. La `class` necesita que lo escribas (salvo que todas sus propiedades tengan valor inicial).
- Solo las `class` tienen **herencia**.

```
struct Punto { var x: Int }
var a = Punto(x: 1)
var b = a          // fotocopia
b.x = 99
print(a.x)         // 1 ← a no cambió
```

P27. ¿Cuándo usas `struct` y cuándo `class`?

`struct` **por defecto** (datos, fichas, vistas de SwiftUI). `class` cuando necesitas **un solo objeto compartido** que sobreviva y que varias partes vean igual (por ejemplo, un `ViewModel`), o cuando necesitas herencia.

P28. ¿Qué es `init` y qué es `self`?

`init` es lo que se ejecuta **al nacer** el objeto: le da sus valores iniciales. `self` es "yo mismo", el objeto actual. En `self.marca = marca`, la de la izquierda es la propiedad del objeto y la de la derecha es el dato que entró por el paréntesis.

```
class Vehiculo {
    var marca: String
    init(marca: String) {
        self.marca = marca
    }
}
```

1.7 Protocolos

P29. ¿Qué es un protocolo?

Un **contrato**: dice qué funciones o propiedades **debe tener** un tipo, sin decir cómo se hacen. El tipo que lo adopta está **obligado** a cumplirlo todo, o no compila.

```
protocol Calculable {  
    func area() -> Double  
}  
struct Cuadrado: Calculable {  
    var lado: Double  
    func area() -> Double { return lado * lado }  
}
```

P30. ¿Qué diferencia hay entre un protocolo y un struct?

El protocolo es **el contrato** (qué debe tener). El struct es **quien lo firma** y lo cumple. Se firma con `:`
→ `struct Cuadrado: Calculable`.

P31. ¿Un tipo puede adoptar varios protocolos? Dame un ejemplo que hayas usado.

Sí, separados por coma: `struct Usuario: Codable, Identifiable`. Ya lo usaste en SwiftUI.

P32. ¿Qué es Protocol-Oriented Programming?

La forma en que Swift prefiere organizar el código: definir comportamientos con **protocolos** y que los structs los adopten, en vez de heredar de clases. SwiftUI funciona así: toda vista firma el protocolo `View`.

Parte 2 — Funciones a fondo

P33. ¿Cuáles son las partes de una función?

```
func sumar(a: Int, b: Int) -> Int {  
  // 👉 👉 parámetros 👉 tipo de lo que sale  
  // nombre  
  return a + b  
}
```

Nombre, parámetros (lo que **entra**, cada uno `nombre: Tipo`), `->` con el tipo de lo que **sale**, y el cuerpo entre `{ }`.

P34. Una función tiene 2 momentos. ¿Cuáles?

1. **Crearla** 🏗️ → `func triple(numero: Int) -> Int { ... }`
2. **Usarla** (llamarla) 🕒 → `triple(numero: 4)` Al crearla, los parámetros están vacíos. Los datos reales se ponen al usarla.

P35. ¿Qué hace el `_` delante de un parámetro?

Hace que **al usar la función no tengas que escribir la palabra**.

```
func triple(numero: Int) -> Int // se usa: triple(numero: 4)  
func triple(_ numero: Int) -> Int // se usa: triple(4)
```

Adentro de la función se sigue usando `numero` en los dos casos.

P36. ¿Qué pasa si pones solo `_` sin nombre: `func triple(_: Int)` ?

Adentro no puedes usar el dato, porque no tiene nombre → *cannot find 'numero' in scope*. Hacen falta las dos partes: `_ numero`.

P37. ¿Qué es el nombre externo y el nombre interno?

En `func saludar(a persona: String)`:

- `a` es el **externo**, el que escribes al usarla → `saludar(a: "Luis")`
- `persona` es el **interno**, el que usas adentro → `print("Hola \ (persona)")` Si escribes **una sola palabra**, esa hace los dos trabajos. El `_` es un externo "vacío".

P38. ¿Si uso `_`, cómo sabe Swift qué dato va en qué parámetro?

Por la **posición**: el primer dato va al primer parámetro, el segundo al segundo. Por eso, cuando hay varios parámetros, conviene ponerles palabras, para que se entienda cuál es cuál:
`contarMayores([5, 12], limite: 10)`.

P39. ¿Qué es `return` de verdad?

La **salida** de la máquina. Los datos **entran** por el paréntesis y el resultado **sale** por el `return`. En el `return` va **el nombre de la caja que llenaste**, nunca un dato escrito a mano, ni el nombre de la función.

P40. ¿Cuál es la diferencia entre `return` y `print`?

`return` **entrega** el resultado a quien llamó la función, para que lo use. `print` solo **muestra** algo en la consola. La función no sabe que existe una pantalla: solo fabrica y entrega.

P41. ¿Qué pasa si hay un `return` dentro de un `for`?

Es una **salida de emergencia** 🚒 : sale de la **función entera** al instante, no solo del `for`.

P42. Si una función dice `-> Int`, ¿qué está obligada a hacer?

A terminar **siempre** con un `return` de tipo `Int`. Si no $\rightarrow$ *missing return in global function expected to return 'Int'*.

P43. ¿Cuándo una función NO lleva `->`?

Cuando solo **hace algo** y no entrega ningún resultado. Por ejemplo `func marcarHoy()` : cambia datos, pero no devuelve nada.

P44. ¿Puedes modificar un parámetro dentro de la función?

No: los parámetros son constantes (como `let`). Si necesitas cambiarlo, haz una copia: `var n = numero`.

Parte 3 — Algoritmos estilo LeetCode

P45. ¿Qué es LeetCode y por qué lo usan en entrevistas?

Una web de problemas de programación por dificultad (Easy, Medium, Hard). Te dan la **firma** de una función y tú escribes el cuerpo. Las empresas lo usan para ver **cómo razonas** un algoritmo, no si sabes armar una app.

P46. ¿Cuál es el molde del acumulador? Dibújalo.

```
func contarPares(_ lista: [Int]) -> Int {
    var total = 0           // 1. la caja: VAR, arriba
    for numero in lista {   // 2. recorrer (otro nombre, no el de la caja)
        if numero % 2 == 0 { // 3. preguntar
            total += 1       // 4. sumar a la caja
        }
    }
    return total           // 5. devolver la caja, AL FINAL, fuera del for
}
```

Las 3 trampas: la caja es `var` (no `let`), el `for` usa otro nombre, y el `return` va al final, fuera del `for`.

P47. ¿Por qué la caja de un acumulador que multiplica empieza en 1 y no en 0?

Porque $0 \times \text{cualquier cosa} = 0$, y arruinaría todo. Para sumar se parte en 0; para multiplicar (como el factorial), en 1.

P48. ¿Qué significa `total += 1`?

Es la forma corta de `total = total + 1`.

P49. En una búsqueda ("¿está el número en la lista?"), ¿dónde va cada `return`?

Para decir SÍ basta encontrar UNO; para decir NO hay que haber mirado TODOS. Entonces el `return true` va **dentro** del `if` (salida de emergencia apenas lo encuentras) y el `return false` va **fuera** del `for`, sin `else`.

```
func contiene(_ lista: [Int], _ buscado: Int) -> Bool {
    for numero in lista {
        if numero == buscado {
            return true
        }
    }
    return false
}
```

P50. ¿Qué es un caso borde (edge case)? Da un ejemplo tuyo.

Un valor raro de los extremos donde la solución general falla: el 0, el 1, la lista vacía, un número negativo. Se protege **arriba del todo** con un `return` temprano. Ejemplo: en `esPrimo`, con 1 y 2 el rango `2... (numero - 1)` queda invertido y crashea. Solución: `if numero <= 2 { ... }` al principio.

P51. ¿Qué preguntas haces antes de empezar a escribir en una entrevista?

1. ¿Qué entra y qué sale? (tipos)
2. ¿Qué pasa con la lista vacía, el 0, los negativos? (casos borde)
3. Dame un ejemplo con su resultado esperado. Y luego **explica tu plan en voz alta antes de escribir**.

P52. ¿Cómo recorres un texto letra por letra? ¿Qué tipo tiene cada letra?

Con `for letra in texto`. Cada letra es un `Character`. Para pegarla a un `String` hay que convertirla: `String(letra)`.

P53. ¿Cómo inviertes un texto?

Acumulador con la letra **adelante**: `resultado = String(letra) + resultado`. Cada letra nueva empuja a las anteriores hacia atrás.

P54. ¿Cómo sabes si un texto es palíndromo?

Primero lo inviertes **completo** (el `for` termina) y **después**, fuera del `for`, comparas `texto == invertido`.

P55. ¿Cómo sumas los dígitos de 123?

Con `while n > 0`: `n % 10` saca el último dígito (3) y lo sumas; `n / 10` lo borra (queda 12). Repites hasta que `n` llega a 0.

P56. Explica Two Sum. ¿Qué devuelve?

Dada una lista y un `target`, encontrar los **dos números que suman el target** y devolver sus **posiciones**, no los valores. `[2, 7, 11, 15]` con `target 9` → `[0, 1]`. Solución con dos `for` anidados: `i` recorre todo y `j` empieza en `i + 1`, para no repetir pares ni sumar un número consigo mismo.

P57. Explica tu algoritmo de la racha más larga (Reto 18).

Dos cajas: `rachaActual` (sube con cada  y vuelve a 0 con cada ) y `mejorRacha` (guarda el récord). En cada : `rachaActual += 1` y, si supera a `mejorRacha`, la reemplaza. Se devuelve `mejorRacha`, **no** `rachaActual`: si el último día falló, `rachaActual` vale 0.

P58. ¿Qué es un error de lógica y qué es un error de sintaxis?

Sintaxis: un símbolo mal escrito (una llave, una coma, un typo). El compilador te lo marca en rojo.
Lógica: compila, pero hace otra cosa (por ejemplo devuelve 1 en vez de 10). Es más difícil, porque nadie te avisa: lo encuentras probando.

P59. ¿Por qué un error en una función impide que corra todo el archivo?

Porque Swift **compila todo el archivo junto** antes de ejecutar. Un eslabón roto detiene toda la cadena.

Parte 4 — SwiftUI

4.1 Vistas y diseño

P60. ¿Qué es una vista en SwiftUI?

Un `struct` que firma el protocolo `View` y tiene una propiedad `body` que dice **qué se dibuja**.

```
struct TarjetaView: View {
    var body: some View {
        Text("Hola")
    }
}
```

Conecta con las lecciones 7 y 8: es un **struct** que cumple un **protocolo**.

P61. ¿Qué significa `some View`?

"Devuelvo **alguna** vista concreta, pero no te digo cuál exactamente". Swift sabe el tipo real, pero no te obliga a escribirlo (sería larguísimo).

P62. ¿SwiftUI es declarativo o imperativo? ¿Qué significa?

Declarativo: describes **cómo debe verse** la pantalla según los datos, y SwiftUI se encarga de dibujarla. Cuando los datos cambian, la pantalla se redibuja sola. (Imperativo, como UIKit, es dar órdenes paso a paso: "cambia este texto, mueve este botón".) Tu analogía: **los datos son los cimientos y ladrillos; la View es la pintura**, lo que se ve.

P63. ¿Qué son `VStack`, `HStack`, `ZStack` y `Spacer`?

- `VStack` → apila **vertical** (uno debajo del otro)
- `HStack` → apila **horizontal** (uno al lado del otro)
- `ZStack` → uno **encima** del otro (capas)
- `Spacer()` → un resorte que empuja y ocupa el espacio libre

P64. ¿Qué es un modificador y dónde va?

Una función que cambia cómo se ve una vista: `.font(.title)`, `.padding()`, `.foregroundColor(.orange)`. Va **después de la }** de lo que modifica. El **orden importa**: cada modificador envuelve al anterior.

4.2 Estado

P65. ¿Qué es `@State`?

Una variable que, **cuando cambia, redibuja la pantalla**. SwiftUI la guarda fuera del struct, así que sobrevive aunque la vista se recree. Se marca `private` porque pertenece solo a esa vista.

```
@State private var contador = 0
Button("Sumar") { contador += 1 } // la pantalla se redibuja sola
```

P66. ¿Por qué una variable normal (sin @State) no sirve para un contador?

Porque la vista es un `struct` que SwiftUI **destruye y recrea** constantemente. Una variable normal se perdería en cada redibujado, y además no avisaría a la pantalla que cambió.

P67. ¿Qué es un Binding y qué significa el \$?

Un **cable de ida y vuelta** hacia una variable `@State`. Con `$nuevoHabit`, el `TextField` puede **leer y escribir** esa variable: lo que el usuario tipea queda guardado en ella.

```
@State private var nuevoHabit = ""
TextField("Escribe un hábito", text: $nuevoHabit)
```

P68. ¿Qué es .toggle() ?

Un interruptor  para un `Bool`: si es `true` lo vuelve `false`, y al revés. `marcados[i].toggle()`.

4.3 Listas y navegación

P69. ¿Qué son List y ForEach ?

`List` es la pantalla con filas (con scroll). `ForEach` recorre un array y **crea una vista por cada elemento**, como un `for` pero para pantallas.

```
List {
    ForEach(habitos) { habit in
        Text(habit.nombre)
    }
}
```

P70. ¿Qué es Identifiable y por qué ForEach lo necesita?

Un protocolo que obliga a tener un `id` único: **el RUT de cada fila**. SwiftUI lo usa para saber qué fila es cuál cuando la lista cambia (qué agregar, borrar o mover).

```
struct Habit: Identifiable {
    let id = UUID() // RUT único inventado por Swift
    let nombre: String
}
```

P71. ¿Qué problema tiene ForEach(lista, id: \.self) ?

Usa el **valor mismo** como RUT. Si hay dos elementos iguales (dos hábitos "Leer"), tienen el mismo RUT y SwiftUI se confunde. `UUID()` lo evita.

P72. ¿Qué es el scope? ¿Por qué da "Cannot find 'habit' in scope"?

El scope es **dónde existe** una variable. `habito` solo existe **dentro de las llaves** del `ForEach { habito in ... }`. Si lo usas afuera, Swift no lo encuentra (igual que el `num` de un `for`).

P73. ¿Cómo navegas a una pantalla de detalle pasando datos?

Con `NavigationStack` (la pila de pantallas) y `NavigationLink` (la fila que se puede tocar). Los datos se pasan por el `init` de la vista de destino.

```
NavigationStack {
    List {
        ForEach(habitos) { habito in
            NavigationLink {
                DetalleView(habito: habito)           // a dónde va
            } label: {
                Text(habito.nombre)                   // cómo se ve la fila
            }
        }
    }
    .navigationTitle("Mis hábitos")
}

struct DetalleView: View {
    let habito: Habito                               // lo que recibe
    var body: some View { Text(habito.nombre) }
}
```

4.4 JSON, errores e internet

P74. ¿Qué es una API y qué es JSON?

La **API** es el mesero entre tu app y el servidor: le pides algo y te lo trae. El plato que trae es **JSON**: texto con formato `{ "clave": valor }` que cualquier lenguaje entiende.

P75. ¿Qué es `Codable` ?

Un protocolo que le da a tu tipo un **pasaporte**  para traducirse **a JSON y desde JSON**. (`Codable` = `Encodable` + `Decodable` .) Los nombres de las propiedades **tienen que ser iguales** a las claves del JSON. El nombre del struct lo inventas tú.

```
struct Tarea: Codable, Identifiable {
    let id: Int
    let title: String           // igual que en el JSON
    let completed: Bool
}
```

P76. ¿Qué hacen `JSONDecoder` y `JSONEncoder` ?

`JSONDecoder` traduce **JSON → tus structs**: `try JSONDecoder().decode([Tarea].self, from: datos)`. `JSONEncoder` traduce **tus structs → JSON**: `try JSONEncoder().encode(estados)`. El `[Tarea].self` le dice al traductor **en qué molde** debe meter el JSON.

P77. ¿Qué es `Data` y para qué sirve `Data(json.utf8)` ?

`Data` son bytes crudos (números). El traductor lee bytes, no texto, así que `Data(json.utf8)` convierte el texto en bytes.

P78. ¿Qué es `try` ? ¿Cuál es la diferencia entre `try` , `try?` y `try!` ?

`try` marca una línea que **puede fallar** (lanzar un error).

- `try` → si falla, el error se maneja en un `do { } catch { }`.
- `try?` → si falla, devuelve `nil` y la app sigue. No te dice qué falló.
- `try!` → si falla, **crash**. Se evita (igual que `!` en los Optionals).

```
do {
    let tareas = try JSONDecoder().decode([Tarea].self, from: datos)
} catch {
    print("Falló: \(error)")    // plan B
}
```

P79. ¿Qué es `async` / `await` ? Explícalo con tu analogía.

El **delivery de pizza** 🍕 :

- `async` = esta función **tarda** (como pedir comida).
- `await` = **espero** aquí hasta que llegue, sin congelar la app.
- `try` = el pedido **puede fallar**.
- `catch` = el **plan B** si falla.
- `URLSession.shared` = el **repartidor compartido** de la app.
- `let (datos, _) =` llega la comida (`datos`) y la boleta (la respuesta), que ignoramos con `_`.

P80. Escribe una función que descargue una lista de internet.

```
func descargarTareas() async -> [Tarea] {
    let url = URL(string: "https://jsonplaceholder.typicode.com/todos")!
    do {
        let (datos, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode([Tarea].self, from: datos)
    } catch {
        return []
    }
}
```

P81. ¿Qué es `.task { }` y por qué no `.onAppear` ?

`.task` ejecuta código **al aparecer la pantalla** y permite usar `await` adentro. Además, si la pantalla desaparece, cancela la tarea sola. `.onAppear` no permite `await` directamente.

```
.task {
    tareas = await descargarTareas()
}
```

P82. ¿Cuáles son los 3 estados de una pantalla que descarga datos?

Cargando, con datos y error / vacío. Una pantalla en blanco sin explicación es mala experiencia. Para el estado vacío usaste `ContentUnavailableView` :

```
.overlay {
    if tareas.isEmpty {
        ContentUnavailableView("Sin Conexión", systemImage: "wifi.slash",
            description: Text("Revisa tu internet e intenta de nuevo"))
    }
}
```

4.5 Arquitectura MVVM

P83. ¿Qué es MVVM?

Separar la app en 3 partes:

- **Model** → los datos (`struct Usuario` , `enum EstadoDia`).
- **View** → la pantalla: **solo dibuja** y le avisa al cerebro cuando el usuario toca algo.
- **ViewModel** → el **cerebro** 🧠 : guarda los datos y tiene la lógica (`marcarHoy()` , `cargar()`). Tu analogía: el **comedor** (View) y la **cocina** (ViewModel). El mesero no cocina.

P84. ¿Por qué el ViewModel es una `class` y no un `struct` ?

Porque la View es un `struct` que SwiftUI **destruye y recrea** todo el tiempo. El cerebro tiene que **sobrevivir** a esos redibujados y ser **un solo objeto compartido**. Una `class` (tipo por referencia) es un solo objeto al que todos apuntan.

P85. ¿Qué hace `@Observable` ?

Convierte la class en algo que la pantalla **observa**: cuando cambia una de sus propiedades, la vista que la usa se redibuja sola. (Es de iOS 17; antes se usaba `ObservableObject` con `@Published` .)

P86. ¿Cómo se conecta la View con el ViewModel?

```
@Observable
class RetoViewModel {
    var hoy = 0
    func marcarHoy() { hoy += 1 }
}

struct RetoView: View {
    @State private var vm = RetoViewModel() // aquí nace el cerebro
    var body: some View {
        Button("Marcar hoy") { vm.marcarHoy() } // "pídele al cerebro"
    }
}
```

`@State` hace que el cerebro nazca **una sola vez** y no se pierda al redibujar.

P87. ¿Qué ventajas tiene MVVM en una entrevista?

1. La lógica se puede **probar** (tests) sin pantalla.
2. La vista queda corta y fácil de leer.
3. Varias pantallas pueden compartir el mismo cerebro.

4.6 Grillas e interacción

P88. ¿Qué es `LazyVGrid` ?

Una **grilla** (cuadrícula): le dices cuántas columnas y va acomodando los elementos en filas. Así hiciste los 66 círculos en 11 columnas × 6 filas. **Lazy** = "perezosa": solo crea las celdas que **se ven en pantalla**, y por eso es rápida con muchos elementos.

```
let columns = Array(repeating: GridItem(.flexible()), count: 11)

LazyVGrid(columns: columns, spacing: 10) {
    ForEach(0..<66, id: \.self) { i in
        Circle().fill(colorDelDia(i))
    }
}
```

P89. ¿Qué es `GridItem(.flexible())` y por qué se repite?

Cada `GridItem` es **una columna**. `.flexible()` = se estira para ocupar el espacio. `Array(repeating: ..., count: 11)` crea 11 columnas iguales (la caja de huevos 🥚).

P90. ¿Qué hace `{ i in ... }` ?

Es un **closure** (una función sin nombre). `i` es el número de esta vuelta: el `ForEach` lo llama una vez por cada número del rango.

P91. ¿Cómo haces que un círculo se pueda tocar?

Con `.onTapGesture { } : Circle().onTapGesture { marcados[i].toggle() } .`

4.7 Persistencia (guardar datos)

P92. ¿Qué es `UserDefaults` ? ¿Para qué NO se usa?

Una **libreta** 📓 de la app: guarda datos chicos con un nombre (clave), y sobreviven aunque cierres la app. `UserDefaults.standard.set(5, forKey: "hoy")` → anota 5 en la página "hoy". `UserDefaults.standard.integer(forKey: "hoy")` → lee esa página. **No se usa** para datos grandes (para eso `SwiftData` o archivos) ni para datos secretos como contraseñas (para eso está el **Keychain**).

P93. ¿Qué es `@AppStorage` ?

Un atajo de SwiftUI: una variable que se guarda **automáticamente** en `UserDefaults` y además redibuja la pantalla, como `@State`. `@AppStorage("hoy") private var hoy = 0 .`

P94. ¿Qué es `didSet` ?

Código que se ejecuta **solo, cada vez que la variable cambia**. Es un timbre 🔔. Se llama *property observer*.

```
var hoy = 0 {  
    didSet { UserDefaults.standard.set(hoy, forKey: "hoy") }  
}  
hoy = 3    // suena el timbre → se anota 3 en la libreta
```

Existe también `willSet`, que suena **justo antes** del cambio.

P95. ¿Por qué no puedes guardar un `[EstadoDia]` directo en `UserDefaults`? ¿Cómo lo resolviste?

Porque la libreta solo entiende tipos simples (`Bool`, `Int`, `String`, `Data`, listas de esos). Lo resolviste en 3 piezas:

1. `enum EstadoDia: Codable` → el pasaporte.
2. `didSet` que traduce a JSON con `JSONEncoder` y lo guarda como `Data`.
3. `init()` que al nacer el cerebro lee la libreta y traduce de vuelta con `JSONDecoder`.

```
var estados: [EstadoDia] = Array(repeating: .pendiente, count: 66) {  
    didSet {  
        let datos = try? JSONEncoder().encode(estados)  
        UserDefaults.standard.set(datos, forKey: "estados")  
    }  
}  
  
init() {  
    if let datos = UserDefaults.standard.data(forKey: "estados"),  
        let guardados = try? JSONDecoder().decode([EstadoDia].self, from: datos) {  
        estados = guardados  
    }  
}
```

4.8 Enum y switch

P96. ¿Qué es un `enum` ?

Un tipo con una **lista cerrada de opciones**. Como un semáforo 🚦: siempre está en uno de sus colores, nunca en dos a la vez, y no existen otros.

```
enum EstadoDia {  
    case pendiente  
    case cumplido  
    case fallado  
    case descanso  
}
```

P97. ¿Qué es cada `case` ?

Cada una de las opciones posibles del enum.

P98. ¿Por qué un `enum` es mejor que dos listas de `Bool` (días y fallados)?

1. Con dos `Bool` un día podría estar **cumplido y fallado a la vez**, y eso es un bug. Con el `enum` es **imposible**: cada día tiene un solo estado.
2. `.cumplido` se entiende solo; `true` no dice qué significa.
3. Es una sola lista en vez de dos. En entrevista se dice: **"haz que los estados imposibles no se puedan representar"**.

P99. ¿Qué significa `.pendiente` con el punto?

La forma corta de `EstadoDia.pendiente`. Swift ya sabe que es un `EstadoDia`, así que basta el punto.

P100. ¿Qué es un `switch`?

Un `if` con varias opciones: mira una caja y ejecuta el `case` que coincide.

```
func colorDelDia(_ i: Int) -> Color {
    switch estados[i] {
    case .pendiente: return .gray.opacity(0.25)
    case .cumplido:  return .orange
    case .fallado:   return .red
    case .descanso:  return .blue
    }
}
```

P101. ¿Qué significa "switch must be exhaustive"?

Exhaustive = **completo**. El `switch` tiene que cubrir **todas** las opciones del `enum`. Si agregas `case descanso` y no lo manejas, **no compila**. Esa es la gran ventaja: el compilador te **obliga** a pensar en el caso nuevo. Con `if` y `Bool` se te olvidaría en silencio.

P102. ¿Cuándo usarías `default` en un `switch`?

Para decir "cualquier otro caso". Con un `enum` propio conviene **no usarlo**: si lo pones, cuando agregues un `case` nuevo el compilador ya no te avisará.

Parte 5 — Encuentra el bug 🐛

Cada código tiene errores. Escribe **en qué línea** está cada uno y **por qué**. Las soluciones están en la Parte 7.

Bug 1 — debería contar los días de sol (3 bugs)

```
func contarDiasDeSol(_ dias: [Clima]) -> Int {
    let total = 0
    for dia in dias {
        if dia = .sol {
            total += 1
            return total
        }
    }
}
```

Bug 2 — debería sumar del 1 al n (1 bug)

```
func sumarHasta(n: Int) -> Int {
    var suma = 0
    for suma in 1...n {
        suma += 1
    }
    return suma
}
```

Bug 3 — debería contar los días fallados (1 bug)

```
func diasFallados() -> Int {
    var caja = 0
    for i in caja {
        if i == true {
            caja += 1
        }
    }
    return caja
}
```

Bug 4 — ¿es primo? (falla con algunos números)

```
func esPrimo(numero: Int) -> Bool {
    for i in 2...(numero - 1) {
        if numero % i == 0 {
            return false
        }
    }
    return true
}
```

Bug 5 — ¿está en la lista? (1 bug de lógica)

```
func contiene(_ lista: [Int], _ buscado: Int) -> Bool {
    for numero in lista {
        if numero == buscado {
            return true
        } else {
            return false
        }
    }
    return false
}
```

Bug 6 — racha más larga (1 bug)

```
func rachaMasLarga(dias: [Bool]) -> Int {
    var rachaActual = 0
    var mejorRacha = 0
    for dia in dias {
        if dia {
            rachaActual += 1
            if rachaActual > mejorRacha {
                mejorRacha = rachaActual
            }
        } else {
            rachaActual = 0
        }
    }
    return rachaActual
}
```

Bug 7 — botón de fallar (1 bug)

```
func fallarHoy() {
    if hoy < 66 {
        estados[hoy] = .cumplido
        hoy += 1
    }
}
```

Bug 8 — función que no compila (1 bug)

```
func triple(_ : Int) -> Int {
    return numero * 3
}
```

Bug 9 — la función hace algo, pero no compila (1 bug)

```
func descansarHoy() -> Int {
    if hoy < 66 {
        estados[hoy] = .descanso
        hoy += 1
    }
}
```

Bug 10 — un ViewModel donde Xcode dice "cannot find 'hoy' in scope" (1 bug)

```
@Observable
class RetoViewModel {
    var hoy = 0
    init() {
        print("nací")
    }
}

func marcarHoy() {
    hoy += 1
}
}
```

Bug 11 — una vista SwiftUI (2 bugs)

```
struct ContadorView: View {
    var contador = 0
    var body: some View {
        Button("Sumar") { contador == contador + 1 }
    }
}
```

Bug 12 — un switch que no compila (1 bug)

```
enum Clima { case sol, lluvia, nieve }

func emoji(_ clima: Clima) -> String {
    switch clima {
        case .sol:    return "☀️"
        case .lluvia: return "☁️"
    }
}
```

Parte 6 — Escribe de memoria 🖋️

Hoja en blanco, sin mirar. Ponte un cronómetro: en una entrevista real tienes entre 15 y 30 minutos por problema. Al terminar, compara con la Parte 7.

1. `contarMayores(_ lista: [Int], limite: Int) -> Int` → cuántos números superan el límite.
`contarMayores([5, 12, 3, 20], limite: 10) → 2`. (*Es tu Reto 19, paso 3.*)
 2. `mayorDeLista(_ lista: [Int]) -> Int` → el número más grande. `[3, 9, 2] → 9`.
 3. `invertir(_ texto: String) -> String` → `"hola" → "aloh"`.
 4. `esPalindromo(_ texto: String) -> Bool` → `"oso" → true`.
 5. `factorial(_ n: Int) -> Int` → `5 → 120`.
 6. `soloPares(_ lista: [Int]) -> [Int]` → `[1, 2, 3, 4] → [2, 4]`.
 7. `twoSum(_ numeros: [Int], target: Int) -> [Int]` → `[2, 7, 11, 15], 9 → [0, 1]`.
 8. `rachaMasLarga(_ dias: [Bool]) -> Int` → `[true, true, false, true, true, true] → 3`.
 9. Un `enum Semaforo` con 3 colores y una función `siguiente(_ s: Semaforo) -> Semaforo` con `switch` (`verde → amarillo → rojo → verde`).
 10. Una vista `SwiftUI` con un `@State` contador, un `Text` que lo muestre y dos botones: "Sumar" y "Reiniciar".
 11. Un `struct Habito: Identifiable` (con `id = UUID()`, `nombre`, `racha`) y una `List` con `ForEach` que muestre nombre y racha en un `HStack` con `Spacer`.
 12. Un `@Observable class` con una lista de `[Bool]` de 66 días, una función `marcarHoy()`, y la vista que lo usa con `@State private var vm`.
-

Parte 7 — Soluciones

Soluciones: Encuentra el bug

Bug 1 — 3 bugs:

1. `let total` → debe ser `var total`: con `let` no se puede hacer `+= 1`.
2. `if dia = .sol` → debe ser `==`: en un `if` se pregunta, no se guarda.
3. `return total` está dentro del `if`: sale en el primer día de sol y devuelve 1. Va **al final, fuera del for**. Además, así como está, la función no tiene `return` si no hay sol.

```
func contarDiasDeSol(_ dias: [Clima]) -> Int {
    var total = 0
    for dia in dias {
        if dia == .sol {
            total += 1
        }
    }
    return total
}
```

Bug 2 — el `for` usa el **mismo nombre** que la caja (`suma`): crea su propia `suma`, que tapa la de arriba, y además una variable del `for` es constante. Debe ser `for i in 1...n { suma += i }`.

Bug 3 — `for i in caja`: `caja` es un número y un número no se recorre. Debe recorrer la lista: `for i in fallados`.

Bug 4 — **caso borde**: con 1 y 2 el rango queda `2...0` o `2...1`, invertido → crash. Hay que protegerlo arriba:

```
if numero < 2 { return false }
if numero == 2 { return true }
```

Bug 5 — el `else { return false }` sale en la **primera** vuelta: si el buscado no es el primer elemento, dice `false` sin mirar el resto. Hay que borrar el `else`: para decir "no" hay que haber mirado **todos**.

Bug 6 — devuelve `rachaActual` en vez de `mejorRacha`. Si el último día es **✗**, devuelve 0.

Bug 7 — guarda `.cumplido` en `fallarHoy()`. Debe ser `.fallado`.

Bug 8 — falta el nombre interno: `func triple(_ numero: Int) -> Int`.

Bug 9 — promete `-> Int` pero no tiene `return`. Esta función solo **hace** algo y no entrega un resultado, así que se borra el `-> Int`.

Bug 10 — hay una `}` de más después del `init`, que **cierra la class antes de tiempo**. `marcarHoy()` queda fuera del cerebro y no encuentra `hoy`. Se borra esa llave.

Bug 11 — 2 bugs:

1. `contador == contador + 1` compara en vez de guardar. Debe ser `contador += 1`.
2. `var contador = 0` → falta `@State private`. Apenas arreglas el bug 1, Xcode reclama *cannot assign to property: 'self' is immutable*: una vista no puede cambiar sus propias variables normales. Con `@State` sí puede, y además la pantalla se redibuja.

Bug 12 — falta `case .nieve`: el `switch` no es exhaustivo (*switch must be exhaustive*).

Soluciones: Escribe de memoria

1. contarMayores

```
func contarMayores(_ lista: [Int], limite: Int) -> Int {
    var total = 0
    for numero in lista {
        if numero > limite {
            total += 1
        }
    }
    return total
}
```

2. mayorDeLista

```
func mayorDeLista(_ lista: [Int]) -> Int {
    var mayor = lista[0]
    for numero in lista {
        if numero > mayor {
            mayor = numero
        }
    }
    return mayor
}
```

Pregunta de entrevista: ¿qué pasa con una lista vacía? `lista[0]` crashea. Se protege con `if lista.isEmpty { return 0 }` (o devolviendo un `Optional Int?`).

3. invertir

```
func invertir(_ texto: String) -> String {
    var resultado = ""
    for letra in texto {
        resultado = String(letra) + resultado
    }
    return resultado
}
```

4. esPalindromo

```
func esPalindromo(_ texto: String) -> Bool {
    var invertido = ""
    for letra in texto {
        invertido = String(letra) + invertido
    }
    return texto == invertido
}
```

`return texto == invertido` devuelve directo el `Bool` de la comparación. Es lo mismo que `if ... { return true } else { return false }`, pero más corto.

5. factorial

```
func factorial(_ n: Int) -> Int {
    var resultado = 1           // empieza en 1, no en 0
    for i in 1...n {
        resultado *= i
    }
    return resultado
}
```

6. soloPares

```
func soloPares(_ lista: [Int]) -> [Int] {
    var pares: [Int] = []
    for numero in lista {
        if numero % 2 == 0 {
            pares.append(numero)
        }
    }
    return pares
}
```

7. twoSum

```
func twoSum(_ numeros: [Int], target: Int) -> [Int] {
    for i in 0..

```

8. rachaMasLarga

```
func rachaMasLarga(_ dias: [Bool]) -> Int {
    var rachaActual = 0
    var mejorRacha = 0
    for dia in dias {
        if dia {
            rachaActual += 1
            if rachaActual > mejorRacha {
                mejorRacha = rachaActual
            }
        } else {
            rachaActual = 0
        }
    }
    return mejorRacha
}
```

9. Semáforo

```
enum Semaforo {
    case verde
    case amarillo
    case rojo
}

func siguiente(_ s: Semaforo) -> Semaforo {
    switch s {
    case .verde:    return .amarillo
    case .amarillo: return .rojo
    case .rojo:     return .verde
    }
}
```

10. Contador

```
struct ContadorView: View {
    @State private var contador = 0

    var body: some View {
        VStack {
            Text("Contador: \(contador)")
            Button("Sumar") { contador += 1 }
            Button("Reiniciar") { contador = 0 }
        }
    }
}
```

11. Lista de hábitos

```
struct Habito: Identifiable {
    let id = UUID()
    let nombre: String
    let racha: Int
}

struct HabitosView: View {
    let habitos = [
        Habito(nombre: "Leer", racha: 5),
        Habito(nombre: "Correr", racha: 12)
    ]

    var body: some View {
        List {
            ForEach(habitos) { habito in
                HStack {
                    Text(habito.nombre)
                    Spacer()
                    Text("🔥 \(habito.racha)")
                }
            }
        }
    }
}
```

12. ViewModel de 66 días

```
@Observable
class RetoViewModel {
    var dias = Array(repeating: false, count: 66)
    var hoy = 0

    func marcarHoy() {
        if hoy < 66 {
            dias[hoy] = true
            hoy += 1
        }
    }
}

struct RetoView: View {
    @State private var vm = RetoViewModel()

    var body: some View {
        Button("✅ Marcar hoy") { vm.marcarHoy() }
    }
}
```

Parte 8 — Lo que viene (todavía no lo vemos)

Preguntas que **seguro** te harán en una entrevista de iOS. Si te las hacen antes de que las estudiemos, di con honestidad: "Lo conozco de nombre y es lo próximo que estoy estudiando".

- **Closures**: ¿qué es un closure? ¿qué significa `@escaping`?
- **ARC y `weak self`**: ¿cómo maneja Swift la memoria? ¿qué es un *retain cycle*?
- **Big O**: ¿por qué tu Two Sum con dos `for` es $O(n^2)$ y cómo se hace en $O(n)$ con un diccionario?
- **Herencia** de classes y `override`.
- **SwiftData**: guardar muchos datos (no solo una libreta).
- **Tests** con XCTest / Swift Testing.
- **Git** en equipo: ramas, pull requests, conflictos.

Mis errores de siempre (léelos antes de cada entrevista)

Error	Ejemplo	Regla
<code>=</code> en vez de <code>==</code>	<code>if i = true</code>	En un <code>if</code> se pregunta → <code>==</code>
<code>let</code> en la caja	<code>let total = 0</code>	La caja que cambia es <code>var</code>
<code>return</code> dentro del <code>for</code>	<code>return total</code> en el <code>if</code>	El <code>return</code> va al final, fuera del <code>for</code>
Mismo nombre en <code>for</code> y caja	<code>for suma in ...</code>	El <code>for</code> usa otro nombre
<code>else</code> colgado de cualquier cosa	<code>return true } else { ...</code>	<code>else</code> solo va pegado al <code>}</code> de un <code>if</code>
Un dato en el <code>return</code>	<code>return [1, 2, 3]</code>	En el <code>return</code> va la caja que llenaste
Llaves descuadradas	<code>}</code> de más que cierra la class	Al abrir <code>{</code> , cierra <code>}</code> al tiro. <code>%A</code> + <code>Ctrl+I</code>
Cortar en vez de copiar	<code>%X</code> borra la línea original	Para duplicar: <code>%C</code> y <code>%V</code>
<code>-> Int</code> sin <code>return</code>	<code>func descansarHoy() -> Int</code>	Si solo hace algo, no lleva <code>-></code>
Recorrer la caja en vez de la lista	<code>for i in caja</code>	El <code>for</code> recorre una lista que ya existe

La lógica la aciertas casi siempre. Lo que falla es lo mecánico, y eso se arregla con repetición.